Recommender Systems (VU) (716.084)

Collaborative Filtering & Matrix Factorization

Denis Helic

ISDS, TU Graz

May 9, 2022

Video (Part 1):
https://courses.isds.tugraz.at/dhelic/kddm1/CF.mp4
Video (Part 2):
https://courses.isds.tugraz.at/dhelic/kddm1/MF.mp4

Outline

1. Introduction
2. Collaborative Filtering
3. User-Based Collaborative Filtering
4. Item-Based Collaborative Filtering
5. Collaborative Filtering: Discussion
6. Data Mining for Recommender Systems
7. Interpolation Weights
8. Factor Analysis
9. Matrix Factorization
10. Theory of Matrix Factorization

Acknowledgments

- Slides are partially based on:
  - Mining Massive Datasets[1] (Stanford University)
  - Gradient descent slides by Prof. Bayen (University of Cal. Berkeley)
  - Collaborative filtering slides by Samer (Graz University of Technology)

[1] http://www.mmds.org

Recommender systems

- Recommender systems predict user responses to options
- E.g. recommend news articles, products, movies, etc. from a large collection of items
- Recommender systems are necessary whenever users interact with a large number of options
- Large catalogs of items: thousands, even millions of products, movies, music, and so on.
- Large number of possible configurations: a personalized luxury car
- Today: **large catalogs of items and large number of users**

Formal model: the utility matrix

Utility Function

Let us denote the set of all users with U and the set of all items with I.
We define the utility function $f : U \times I \to R$, where R is a set of ratings and is a totally ordered set.
For example, $R = \{1, 2, 3, 4, 5\}$ set of star ratings, or $R = [0, 1]$ set of real numbers from that interval.

The Utility Matrix

- The utility function maps user-item tuples to numbers r_{ui}
- We collect these numbers in a utility matrix $\mathbf{R} \in \mathbb{R}^{n \times m}$
- n is the number of users and m the number of items
- If a user rated an item then the utility matrix keeps this rating in the corresponding element
- There are blank elements in the utility matrix in cases where user-item tuples are without a rating

The Utility Matrix

- Typically, the utility matrix is sparse
- This means that most entries are unknown, i.e. most of the entries are blank
- In other words, the majority of user preferences for specific items is unknown
- A missing rating means solely that we do not have explicit information
- It does not mean that the rating is low (or high)
- **The goal of a recommender system is to predict the blank elements from the utility matrix**

The Utility Matrix: example

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Collaborative Filtering (CF)

Collaborative Filtering (CF)

Collaborative Filtering (CF)

- The most prominent approach to generate recommendations
- Used by large, commercial e-commerce sites, e.g. Amazon
- Well-understood, various algorithms and variations exist
- Applicable in many domains (book, movies, music, ..)

Basic Idea

- Use the "wisdom of the crowd" to recommend items
- Users give ratings to catalog items (implicitly or explicitly)
- Customers who had similar tastes in the past, will have similar tastes in the future
- User preferences remain stable and consistent over time

Basic Approach

- A numerical prediction indicating to what degree the current user will like or dislike a certain item
- Top-N list of recommended items
- CF is based on a nearest neighbors approach
- Given a current user (Alice) and a particular item i there are two possibilities:
  1. Find a set of similar users to Alice and look how they liked i (user-based nearest neighbor)
  2. Find a set of similar items to i and look how Alice liked those items (item-based nearest neighbors)

User-Based Collaborative Filtering

User-Based Collaborative Filtering

Basic Approach

- Given a user u and item i not yet rated by u
  1. Find k similar users (neighbors) who liked the same items in the past and who have rated i
  2. Use an aggregation of the neighbors' rating (e.g. mean) to predict r_{ui}
- Iterate over all items i not rated by u
- Recommend Top-N (best rated) items

Similarity and Aggregation

- Three key design decisions:
  1. How to measure similarity?
  2. How many neighbors should we consider (what is k)?
  3. How to predict, i.e. aggregate?
  4. How many recommendations (what is N)?

Collaborative filtering: example

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Similarity: Intuition

- Users A and B are similar
- Users A and C are dissimilar
- Why is that?

Similarity: Intuition

- Users A and B are similar
- Users A and C are dissimilar
- Why is that?
- A and B rated only one movie together, but both gave a high rating
- A and C rated two movies together, but with opposing ratings

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Similarity

- The key is to calculate a similarity score between users u and v: σ_{uv}
- We start by representing user ratings as vectors $\mathbf{r}_u$, i.e, row vectors from the matrix $\mathbf{R}$
- With these vectors we can calculate various similarity scores
- E.g. cosine similarity

Cosine similarity

- Let $\mathbf{r}_u$ be the rating vector of the user u
- We set zeros in blank spaces

$$\mathbf{r}_A^T = \begin{pmatrix} 4 & 0 & 0 & 5 & 2 & 0 & 0 \end{pmatrix}$$
$$\mathbf{r}_B^T = \begin{pmatrix} 5 & 5 & 2 & 0 & 0 & 0 & 0 \end{pmatrix}$$
$$\mathbf{r}_C^T = \begin{pmatrix} 0 & 0 & 0 & 2 & 4 & 5 & 0 \end{pmatrix}$$

Collaborative filtering: Cosine similarity

- The cosine similarity measures the cosine of the angle between the vectors:

$$\begin{aligned} \sigma_{uv} &= \frac{\mathbf{r}_u^T \mathbf{r}_v}{\|\mathbf{r}_u\|_2 \|\mathbf{r}_v\|_2} \\ &= \frac{\sum_{i \in I} r_{ui} r_{vi}}{\sqrt{\sum_{i \in I} r_{ui}^2} \sqrt{\sum_{i \in I} r_{vi}^2}} \end{aligned}$$

Cosine similarity

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

$$\sigma_{AB} = 0.40572$$
$$\sigma_{AC} = 0.4$$

Cosine similarity: Discussion

- This does not capture our intuition that users A and C are dissimilar and users A and B are similar

Cosine similarity: Discussion

- This does not capture our intuition that users A and C are dissimilar and users A and B are similar
- The problem is that we treated blanks as zeros and therefore as negative ratings

Cosine similarity: Discussion

- This does not capture our intuition that users A and C are dissimilar and users A and B are similar
- The problem is that we treated blanks as zeros and therefore as negative ratings
- Alternative: treat zeros as average ratings
- Let us move to the center and calculate correlations, i.e. centered cosine similarity
- While calculating the averages and centering we will ignore the blank spaces

$$\mathbf{x}_u = \mathbf{r}_u - \overline{r_u}$$

Pearson correlation coefficient

$$
\begin{aligned}
\bar{r}_A &= \frac{11}{3} \\
\bar{r}_B &= \frac{12}{3} = 4 \\
\bar{r}_C &= \frac{11}{3}
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{x}_A^T &= \begin{pmatrix} 1/3 & 0 & 0 & 4/3 & -5/3 & 0 & 0 \end{pmatrix} \\
\mathbf{x}_B^T &= \begin{pmatrix} 1 & 1 & -2 & 0 & 0 & 0 & 0 \end{pmatrix} \\
\mathbf{x}_C^T &= \begin{pmatrix} 0 & 0 & 0 & -5/3 & 1/3 & 4/3 & 0 \end{pmatrix}
\end{aligned}
$$

Pearson correlation coefficient

- Now we can calculate correlations, i.e. centered cosine similarity:

$$\begin{aligned}\sigma_{uv} &= \frac{\mathbf{x}_u^T \mathbf{x}_v}{\|\mathbf{x}_u\|_2 \|\mathbf{x}_v\|_2} \\ &= \frac{\sum_{i \in I}(r_{ui} - \overline{r_u})(r_{vi} - \overline{r_v})}{\sqrt{\sum_{i \in I}(r_{ui} - \overline{r_u})^2}\sqrt{\sum_{i \in I}(r_{vi} - \overline{r_v})^2}}\end{aligned}$$

Pearson correlation coefficient

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

$$\sigma_{AB} = 0.06299$$
$$\sigma_{AC} = -0.59524$$

Pearson correlation coefficient: Discussion

- Replaces blank spaces with average user ratings by centering
- Considers “tough” and “easy” raters
- Some people are just more generous than the others
- Already models global effects through user averages
- Typically used in user-based collaborative filtering

Pearson correlation coefficient

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

$$\sigma = \begin{pmatrix} 1. & 0.06299 & -0.59524 & 0.38576 \\ 0.06299 & 1. & 0. & 0.20412 \\ -0.59524 & 0. & 1. & -0.38576 \\ 0.38576 & 0.20412 & -0.38576 & 1. \end{pmatrix}$$

Prediction

- We want to predict the rating of user u for item i (not rated by u)
- We select the neighborhood $N(u;i)$ of user u
- That is k most similar users to user u who have rated item i
- As with similarities we have several possibilities to estimate $\hat{r}_{ui}$

Prediction: first alternative

- Averaging over k-nearest neighbors:

$$\hat{r}_{ui} = \frac{1}{k} \sum_{v \in N(u;i)} r_{vi}$$

Prediction: first alternative

- We want to predict rating of user A for HP2
- We will use the neighborhood $N(u;i)$ of size 2, i.e. $k = 2$
- Two most similar users to user A are B and D, i.e. $N(A;HP2) = \{B, D\}$

$$\sigma = \begin{pmatrix} 1. & 0.06299 & -0.59524 & 0.38576 \\ 0.06299 & 1. & 0. & 0.20412 \\ -0.59524 & 0. & 1. & -0.38576 \\ 0.38576 & 0.20412 & -0.38576 & 1. \end{pmatrix}$$

Prediction: first alternative

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	$\hat{r}_{A,HP2}$		5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

$$\begin{aligned}\hat{r}_{A,HP2} &= \frac{1}{2}\sum_{v\in\{B,D\}} r_{v,HP2} \\ &= \frac{1}{2}(r_{B,HP2} + r_{D,HP2}) \\ &= 4\end{aligned}$$

Prediction: first alternative

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4		5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: first alternative

- To predict Top-N movies we predict ratings for all movies that the user has not rated and pick top N highest predictions

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4	2	5	2	3	3
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: first alternative

- Potential problem:

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4	2	5	2	3	3
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: first alternative

- User C is in $N(A; SW2)$ but $\sigma_{AC} = -0.59524$

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4	2	5	2	3	3
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: second alternative

- Weighted average over k-nearest neighbors:

$$\hat{r}_{ui} = \frac{\sum_{v \in N(u;i)} \sigma_{uv} r_{vi}}{\sum_{v \in N(u;i)} |\sigma_{uv}|}$$

Prediction: second alternative

- We want to predict rating of user A for HP2

$$\begin{aligned}
\hat{r}_{A,HP2} &= \frac{\sum_{v \in \{B,D\}} \sigma_{A,v} \times r_{v,HP2}}{\sum_{v \in \{B,D\}} |\sigma_{A,v}|} \\
&= \frac{0.06299 \times r_{B,HP2} + 0.38576 \times r_{D,HP2}}{0.06299 + 0.38576} \\
&= 3.2807
\end{aligned}$$

Prediction: second alternative

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	3.2807		5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: second alternative

- To predict Top-N movies we predict ratings for all movies that the user has not rated and pick top N highest predictions

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	3.2807	2	5	2	-2.6406	3
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: third alternative

- But some users are "tough" whereas others are "easy" raters
- We may first subtract the neighbors' bias before averaging
- Calculate whether the neighbors' ratings for the unseen item i are higher or lower than their average
- Combine the rating differences

$$\hat{r}_{ui} = \overline{r_u} + \frac{\sum_{v \in N(u;i)} \sigma_{uv}(r_{vi} - \overline{r_v})}{\sum_{v \in N(u;i)} |\sigma_{uv}|}$$

Prediction: third alternative

- We want to predict rating of user A for HP2

$$
\begin{aligned}
\hat{r}_{A,HP2} &= \overline{r_A} + \frac{\sum_{v \in \{B,D\}} \sigma_{A,v} \times (r_{v,HP2} - \overline{r_v})}{\sum_{v \in \{B,D\}} |\sigma_{A,v}|} \\
&= \frac{11}{3} + \frac{0.06299 \times (r_{B,HP2} - 4) + 0.38576 \times (r_{D,HP2} - 2)}{0.06299 + 0.38576} \\
&= 4.6667
\end{aligned}
$$

Prediction: third alternative

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4.6667		5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction: third alternative

- To predict Top-N movies we predict ratings for all movies that the user has not rated and pick top N highest predictions

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	4.6667	1.6667	5	2	2.5271	4.6667
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Further alternatives for improving predictions

- Not all neighbor ratings might be equally valuable
  - Agreement on commonly liked items is not so informative as agreement on controversial items
  - Possible solution: Give more weight to items that have a higher variance
- Number of co-rated items
  - Use significance weighting by e.g. linearly reducing the weight when the number of co-rated items is low
- Case amplification
  - Intuition: Give more weight to highly similar neighbors, i.e., where the similarity value is close to 1
- Neighborhood selection
  - Use similarity threshold

Item-Based Collaborative Filtering

Item-Based Collaborative Filtering

Basic approach

- So far user-based collaborative filtering
- A dual approach is item-based collaborative filtering
- The idea is the same as before but we now switch users and items
- Thus, for an item i we find its neighborhood, which are its most similar items
- Then we estimate rating based on the ratings for the similar items
- For similarities and prediction we can use same approaches as before

Cosine similarity

$$\begin{aligned}\sigma_{ij} &= \frac{\mathbf{r}_i^T \mathbf{r}_j}{\|\mathbf{r}_i\|_2 \|\mathbf{r}_j\|_2} \\ &= \frac{\sum_{u \in U} r_{ui} r_{uj}}{\sqrt{\sum_{u \in U} r_{ui}^2} \sqrt{\sum_{u \in U} r_{uj}^2}}\end{aligned}$$

Cosine similarity

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4			5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

$$\sigma = \begin{pmatrix} 1.00000 & 0.66959 & 0.78087 & 0.58001 & 0.27264 & 0.00000 & 0.00000 \\ 0.66959 & 1.00000 & 0.85749 & 0.00000 & 0.11227 & 0.10090 & 0.51450 \\ 0.78087 & 0.85749 & 1.00000 & 0.00000 & 0.00000 & 0.00000 & 0.00000 \\ 0.58001 & 0.00000 & 0.00000 & 1.00000 & 0.72940 & 0.36418 & 0.00000 \\ 0.27264 & 0.11227 & 0.00000 & 0.72940 & 1.00000 & 0.89872 & 0.21822 \\ 0.00000 & 0.10090 & 0.00000 & 0.36418 & 0.89872 & 1.00000 & 0.19612 \\ 0.00000 & 0.51450 & 0.00000 & 0.00000 & 0.21822 & 0.19612 & 1.00000 \end{pmatrix}$$

Prediction

- Weighted average over k-nearest neighbors:

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i;u)} \sigma_{ij} r_{uj}}{\sum_{j \in N(i;u)} |\sigma_{ij}|}$$

Prediction

- We want to predict rating of user A for HP2
- We will use the neighborhood $N(i;u)$ of size 2, i.e. $k = 2$
- Two most similar items to item HP2 also rated by A are HP1 and SW1, i.e. $N(HP2;A) = \{HP1, SW1\}$

$$\begin{aligned}
\hat{r}_{A,HP2} &= \frac{\sum_{j \in \{HP1,SW1\}} \sigma_{HP2,j} \times r_{A,j}}{\sum_{j \in \{HP1,SW1\}} |\sigma_{HP2,j}|} \\
&= \frac{0.66959 \times r_{A,HP1} + 0.11227 \times r_{A,SW1}}{0.66959 + 0.11227} \\
&= 3.7128
\end{aligned}$$

Prediction

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	3.7128		5	2		
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Prediction

- To predict Top-N movies we predict ratings for all movies that the user has not rated and pick top N highest predictions

User \ Movie	HP1	HP2	HP3	Hobbit	SW1	SW2	SW3
A	4	3.7128	4	5	2	2.8651	2
B	5	5	2				
C				2	4	5	
D		3			1	1	3

Collaborative Filtering: Discussion

Collaborative Filtering: Discussion

Optimization of hyperparameters

- Typically we would apply cross-validation to select the optimal hyperparameters
- Hyperparameters: similarity, prediction formula, etc.
- The size of the neighborhood k
- An analysis of the MovieLens dataset indicates that in most real-world situations, a neighborhood of 20 to 50 neighbors seems reasonable

User-based vs. Item-based

- In theory user-based and item-based are dual approaches
- In practice item-based outperforms user-based
- Items are “simpler” than users
- Items belong to a small set of “genres”, user tastes vary greatly
- Users have multiple interests
- Item similarities are supposed to be more stable than user similarities

Memory-based approaches

- User-based CF is typically memory-based
- The rating matrix is directly used to find neighbors / make predictions
- Does not scale for most real-world scenarios
- Large e-commerce sites have tens of millions of customers and millions of items

Model-based approaches

- Based on an offline pre-processing or model-learning phase
- At run-time, only the learned model is used to make predictions
- Models are updated / re-trained periodically
- Model-building and updating can be computationally expensive
- Item-based CF is typically model-based

Model-based approaches

- Calculate all pair-wise item similarities in advance
- The neighborhood to be used at run-time is typically rather small, because only items are taken into account which the user has rated
- Up to m^2 pair-wise similarities to be memorized in theory
- In practice, this is significantly lower (items with no co-ratings)
- Further reductions possible, e.g. minimum threshold for co-ratings

Problems

- Users not willing to provide (explicit) ratings: collect implicit ratings, e.g. clicks, page views, time spent on page, etc.
- Cold start (not enough users to find a match)
- Sparsity (too many items – some items do not have enough ratings)
- First rater (can not recommend unrated items)
- Popularity bias (Harry Potter effect)

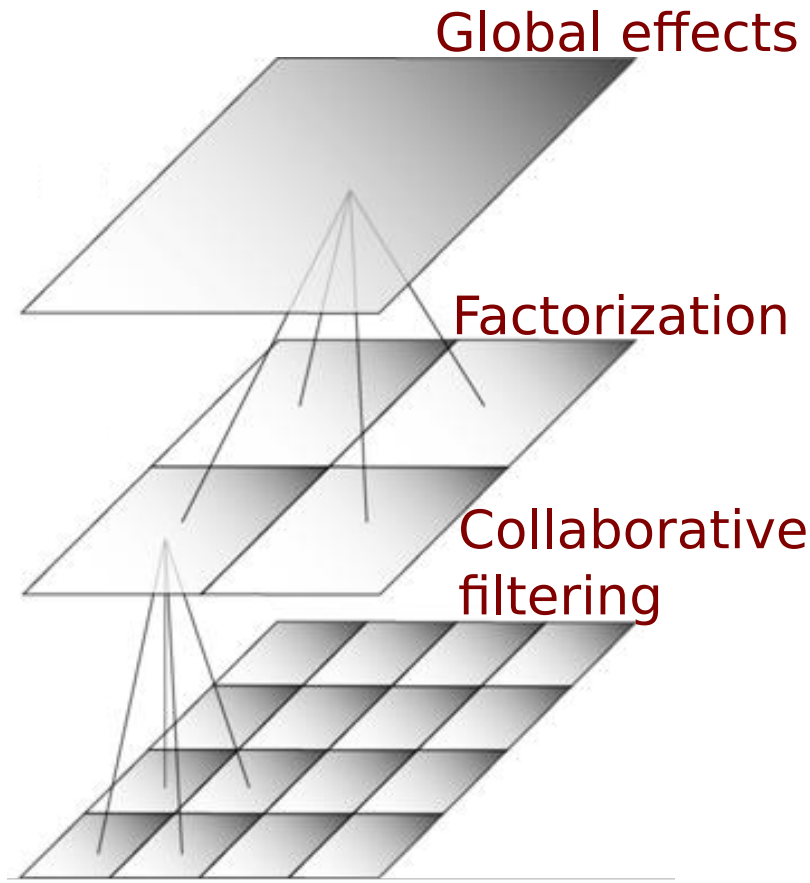
Data Mining for Recommender Systems

Two main steps

1. We want to learn everything you need from the data
   - Classical RecSys (e.g. collaborative filtering): we start with a plausible model and populate the model with data, e.g. a model for calculating similarity
   - Data Mining Approach to RecSys: We learn the items similarity directly from the data
2. We also want to learn at different levels:
   - Global
   - Local
   - Regional

Multi-scale modeling

- Multi-scale modeling of the data
- Global (overall deviations of users/movies)
- Collaborative filtering (extract local patterns)
- Factorization (regional effects)



Modeling global effects

- Mean movie rating $\mu = 3.7$ stars
- The Sixth Sense is $b_i = 0.5$ stars above average
- Joe rates $b_u = -0.2$ stars below average
- Global baseline estimation?

Modeling global effects

- Mean movie rating $\mu = 3.7$ stars
- The Sixth Sense is $b_i = 0.5$ stars above average
- Joe rates $b_u = -0.2$ stars below average
- Global baseline estimation?
- $\hat{r}_{ui} = \mu + b_i + b_u = 3.7 + 0.5 - 0.2 = 4$
- Joe will rate The Sixth Sense 4 stars

Modeling local effects

- Local neighborhood
- Collaborative filtering / nearest neighbors
- We may also want to combine global and local
- E.g. adjust the Joe's global rating with local effects

Running Example: The Netflix Prize

- Training data
  - 100 million ratings, 480,000 users, 17,770 movies
  - 6 years of data: 2000-2005
- Test data
  - Last few ratings of each user (2.8 million)
  - Evaluation criterion: Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{|\mathbf{R}|} \sum_{(u,i) \in \mathbf{R}} (\hat{r}_{ui} - r_{ui})^2}$$

Running Example: The Netflix Prize

- Netflix's system RMSE: 0.9514
- Competition
  - 2,700+ teams
  - $1 million prize for 10% improvement on Netflix

Running Example: The Netflix Utility Matrix **R**

User \ Movie	i1	i2	i3	i4	i5	i6	i7	...	i17700
u1	3					2		...	5
u2			1	2				...	
u3	4							...	
u4				1			3	...	
u5			5	4				...	
u6						1		...	2
u7	1						3	...	
...	...	...	...	...	...	...	...	...	...
u480000								...	2

Running Example: The Netflix Utility Matrix **R**

User \ Movie	i1	i2	i3	i4	i5	i6	i7	...	i17700
u1	3					2		...	5
u2			1	2				...	
u3	4							...	
u4				1			3	...	
u5			5	4				...	
u6						1		...	?
u7	1						?	...	
...	...	...	...	...	...	...	...	...	...
u480000								...	?

Table: Yellow: training dataset, Lightgray: test dataset

$$RMSE = \sqrt{\frac{1}{|\mathbf{R}|} \sum_{(u,i) \in \mathbf{R}} (\hat{r}_{ui} - r_{ui})^2}$$

Interpolation Weights

Interpolation Weights

Local Modeling with Interpolation weights: Idea

- Use a weighted sum rather than weighted average

$$\hat{r}_{ui} = \sum_{j \in N(i;u)} w_{ij} r_{uj}$$

- $N(i;u)$: set of movies rated by user u that are similar to movie i
- w_{ij} is the interpolation weight and we allow $\sum_{j \in N(i;u)} w_{ij} \neq 1$
- w_{ij} models interaction between pairs of movies and it does not depend (only) on user u

Interpolation weights

$$\hat{r}_{ui} = \sum_{j \in N(i;u)} w_{ij} r_{uj}$$

- How to set w_{ij}
- Take a machine learning approach and learn the weights from the data
- The error metric: $RMSE = \sqrt{\frac{1}{|\mathbf{R}|} \sum_{(u,i) \in \mathbf{R}} (\hat{r}_{ui} - r_{ui})^2}$
- Equivalently: $SSE = \sum_{(u,i) \in \mathbf{R}} (\hat{r}_{ui} - r_{ui})^2$

Learning interpolation weights

$$SSE = \sum_{(u,i) \in \mathbf{R}} (\hat{r}_{ui} - r_{ui})^2$$

- Find w_{ij} that minimizes SSE on **training data**
- w_{ij} can be learned based on u and all other users that rated i
- Why is learning on training data a good idea?

Learning interpolation weights

- Goal: make good recommendations
- Quantify goodness using RMSE (SSE)
- Lower RMSE → better recommendations
- Want to make good recommendations on items that user has not yet seen
- We can't see in the future
- Build a system that works well on known ratings
- **Hope** that it will predict well the unknown ratings

Recommendations via optimization

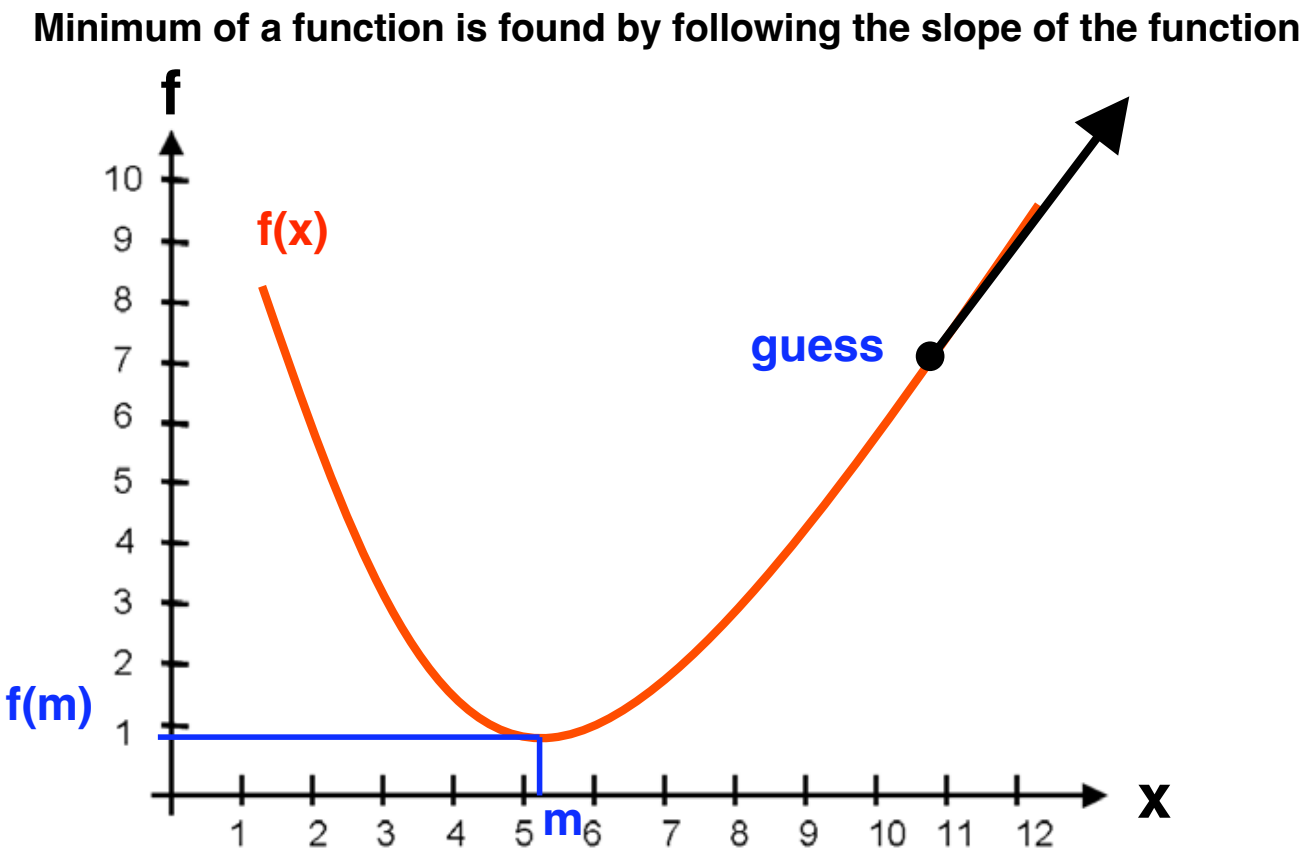
- Let us set values $\mathbf{w}$ such that they work well on known ratings
- How to find such values $\mathbf{w}$?
- Define an objective function and solve the optimization problem
- Find w_{ij} that minimizes SSE on training data

$$J(w) = \sum_{(u,i) \in \mathbf{R}} \sum_{j \in N(i;u)} (w_{ij} r_{uj} - r_{ui})^2$$

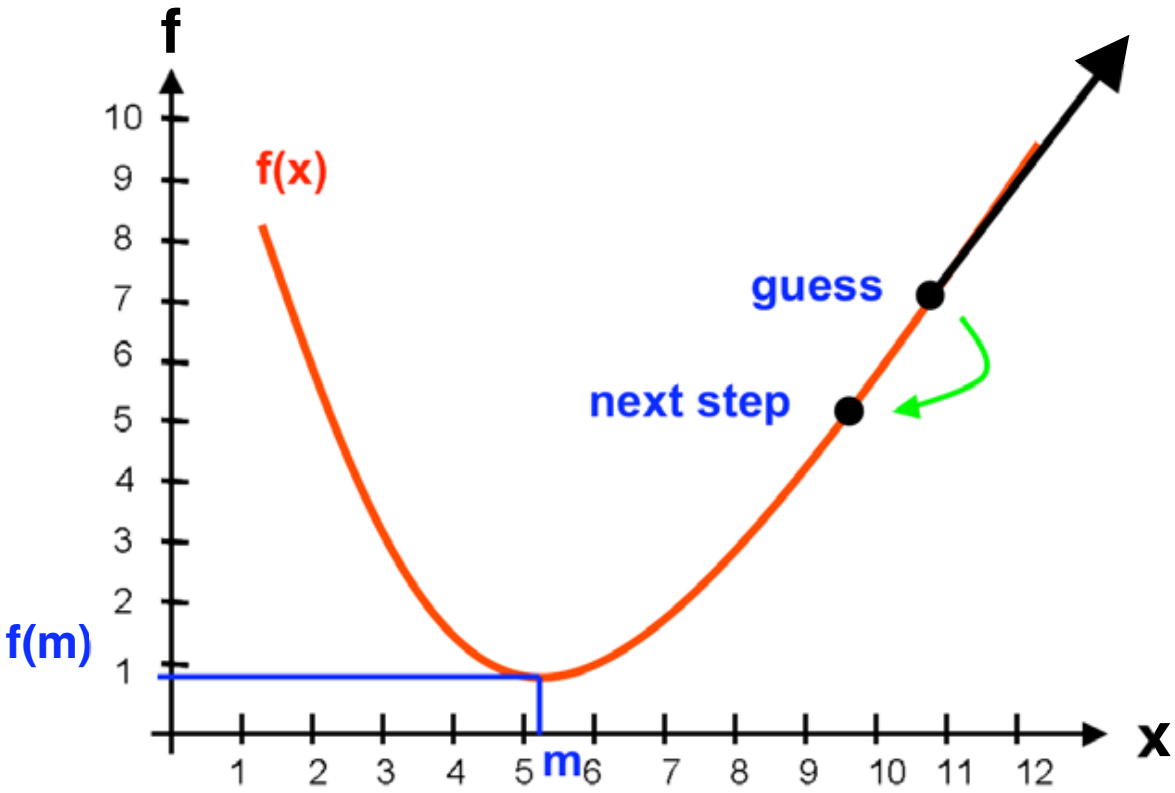
Minimizing a function: gradient descent

- A simple way to minimize a function $f(\mathbf{x})$
- Compute gradient and move in the opposite direction of the gradient $\nabla f(\mathbf{x})$
- $\mathbf{x}_{n+1} = \mathbf{x}_n - \eta \nabla f(\mathbf{x}_n)$
- Repeat until convergence or some criterion is met
- Parameter η is the learning rate

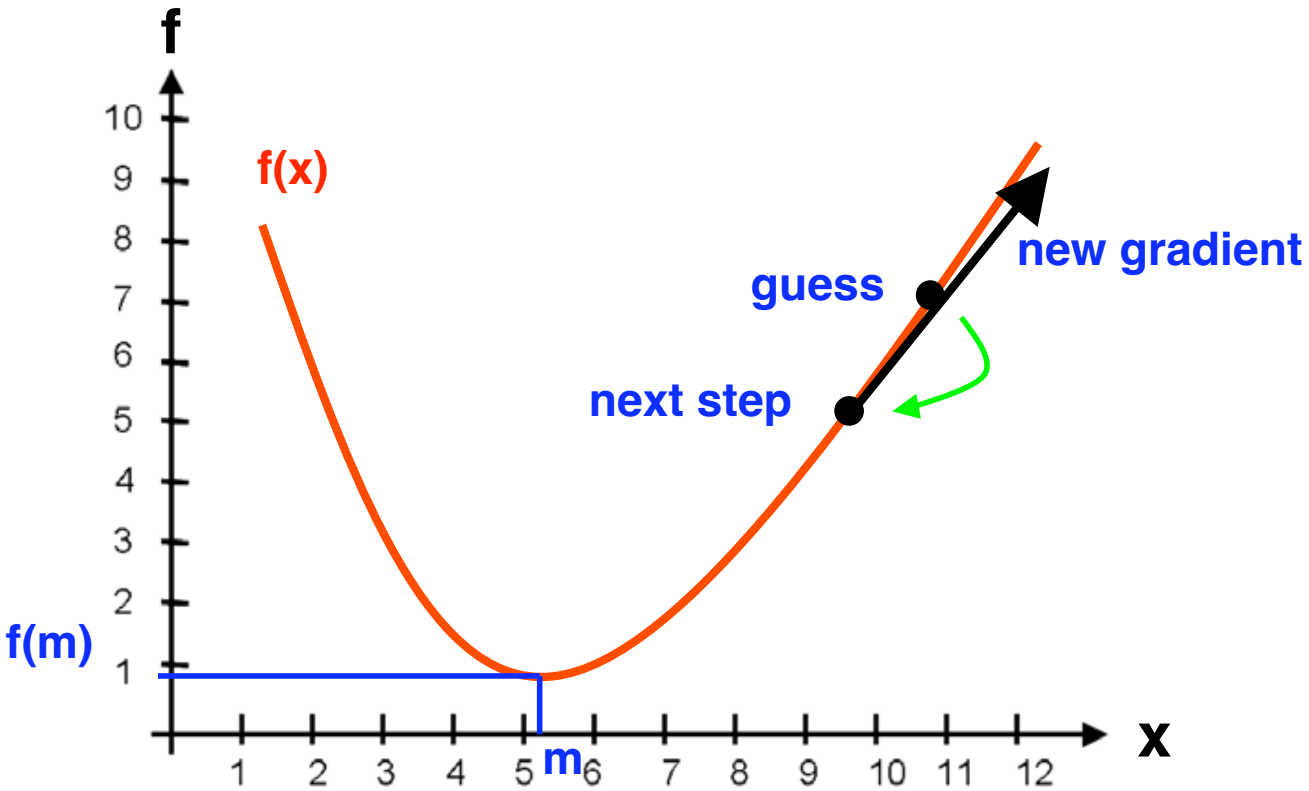
Minimizing a function: gradient descent



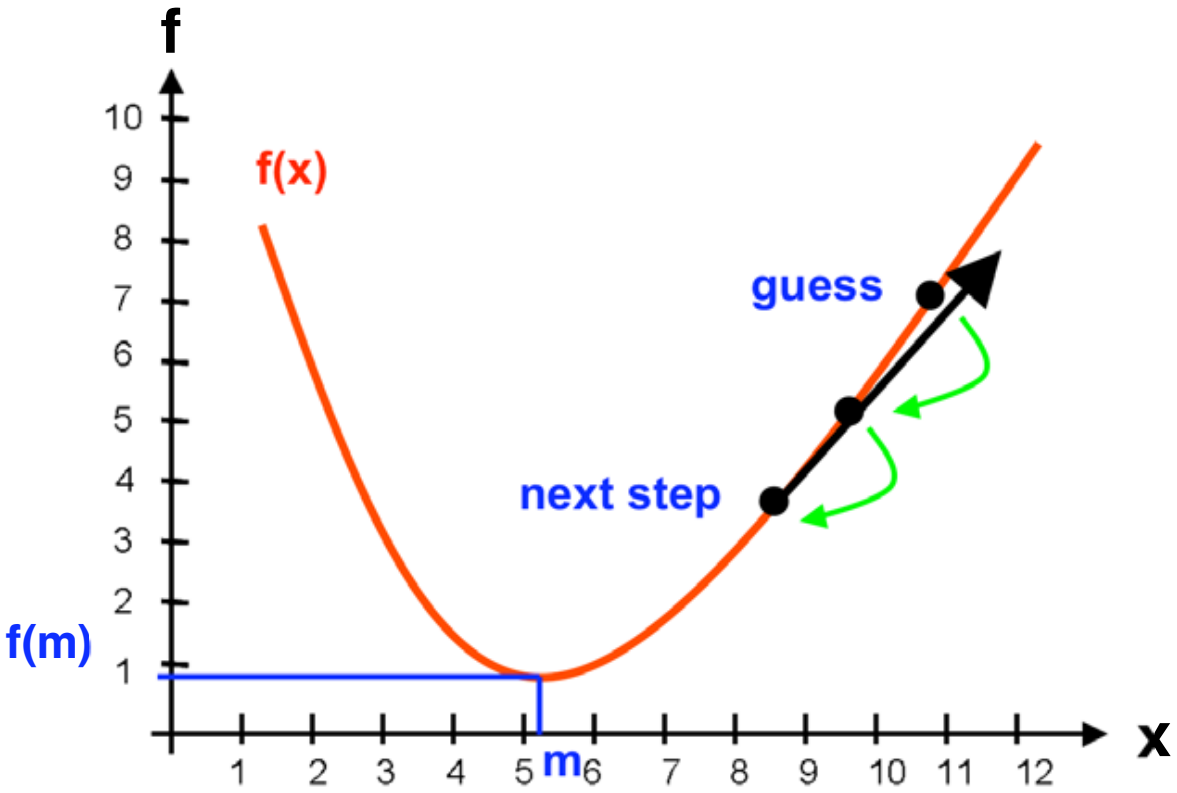
Minimizing a function: gradient descent



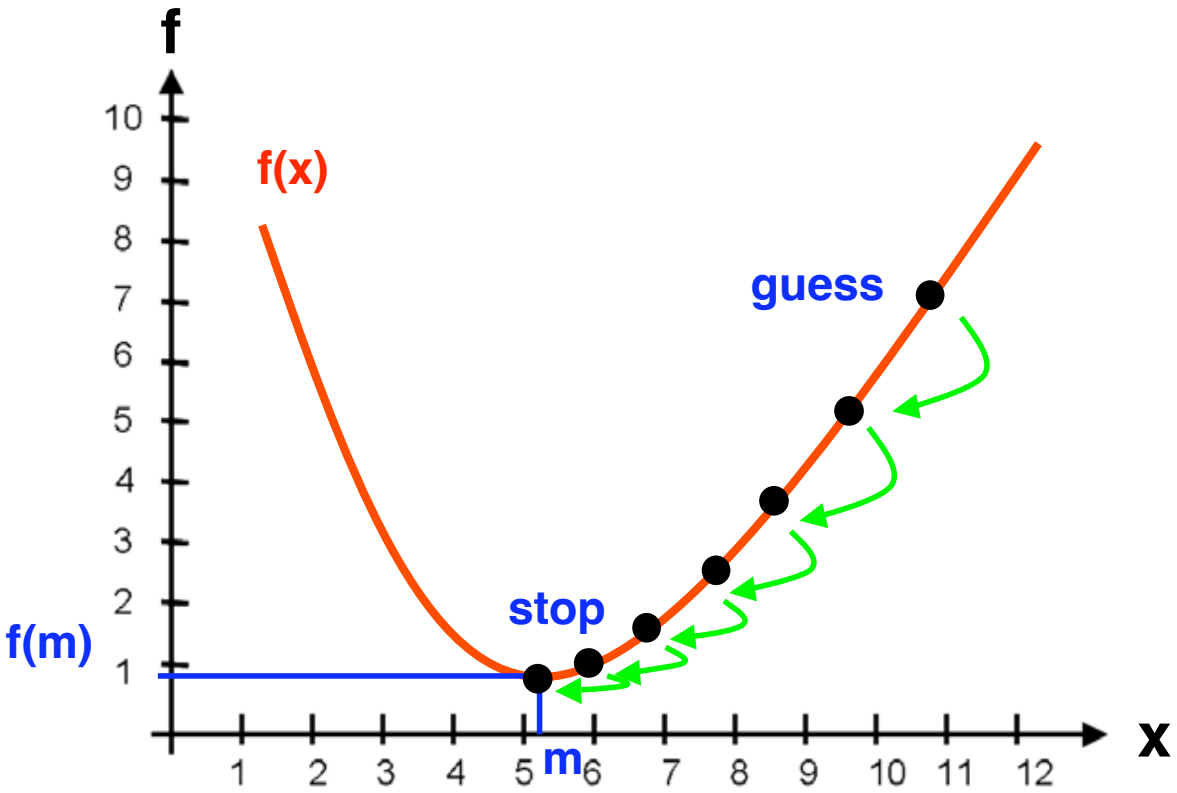
Minimizing a function: gradient descent



Minimizing a function: gradient descent



Minimizing a function: gradient descent



Learning the interpolation weights

$$J(w) = \sum_{(u,i)\in \mathbf{R}} \sum_{j \in N(i;u)} (w_{ij} r_{uj} - r_{ui})^2$$

- Gradient descent
- $\mathbf{w}_{n+1} = \mathbf{w}_n - \eta \nabla_{\mathbf{w}} J(\mathbf{w})$
- η is learning rate
- If too high → problem of overshooting and divergence
- If too low → problem of slow convergence

Learning the interpolation weights

$$\nabla_{\mathbf{w}} J(w) = \left[\frac{\partial J(\mathbf{w})}{\partial w_{ij}} \right] = 2 \sum_{(u,i) \in \mathbf{R}} \sum_{j \in N(i;u)} (w_{ij} r_{uj} - r_{ui}) r_{uj}$$

- We fix movie i, go over all r_{ui}, for every movie $j \in N(i;u)$ and compute $\frac{\partial J(\mathbf{w})}{\partial w_{ij}}$

Regularization

- Typically we want to control the learned parameters w_{ij} and keep them small
- Avoids *overfitting*
- We will interpret this later in the lecture

$$J(w) = \sum_{(u,i)\in \mathbf{R}} \sum_{j \in N(i;u)} (w_{ij} r_{uj} - r_{ui})^2 + \lambda \sum_{j \in N(i;u)} w_{ij}^2$$

The Netflix Prize: Performance of various methods

- Global average: 1.1296
- User average: 1.0651
- Movie average: 1.0533
- Netflix: 0.9514
- CF: 0.94
- CF+Biases+learned weights: 0.91
- Grand Prize: 0.8563

Factor Analysis

Item profiles

- Let us think about the utility matrix as a final result of a rating process
- For example, items have some properties that users like/dislike
- User rate items based on item properties and their preferences for these properties
- For example, movies have technical properties such as color/black-white, resolution, sound, etc.
- Movies have genres such as scifi, romance, action, etc.
- Story telling properties such as linear, retrospective, etc.

Item profiles

- Using item properties we build **item profiles** by assigning numerical values for each property
- A higher value means a stronger presence of the property in a given item
- Thus, an item profile is a **numerical description of an item**
- But, it is difficult to define the properties manually
- The number of properties is high and in most cases we do not know what are the interesting properties

Item profiles

- Also, some of the properties are hidden or latent and can not be expressed directly

  Really dark, slow paced movies with minimal story, but incredible atmosphere, kinda like Drive, The Rover, or No Country for Old Men? [2]

  Movies that will make me want to cry [...] like Extremely Loud and Incredibly Close. [3]

  Movies that take place primarily in one room or building. [4]

[2] https://www.reddit.com/r/MovieSuggestions/comments/3kjrus
[3] https://www.reddit.com/r/MovieSuggestions/comments/11ycep
[4] https://www.reddit.com/r/MovieSuggestions/comments/4va9p8

User profiles

- We use the same set of properties to represent user preferences
- We use numerical values to model the strength of the user preferences
- E.g. a property, which the user prefers gets a higher weight
- The final rating of a user for an item is a weighted sum of the product of the preference and property weights from the item profile
- We can use this idea to predict the missing values in the utility matrix

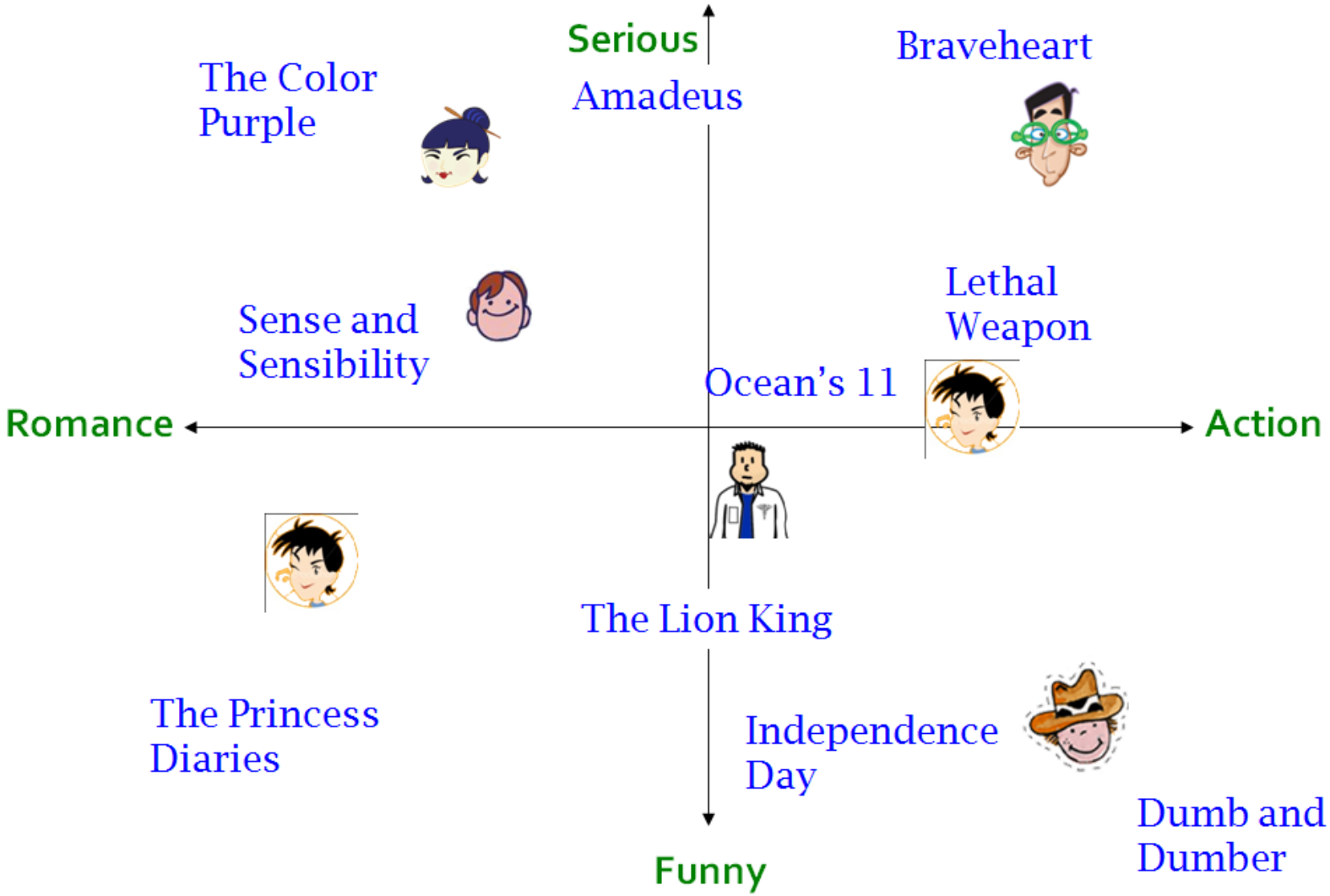
Factors

- **Problem:** But we do not know which properties to use or how to describe them

Factors

- **Problem:** But we do not know which properties to use or how to describe them
- **Solution:** Learn properties from the data
- Terminology: if we learn properties from the data we call them **factors**
- Latent (hidden) factors

Factors


Serious
Funny
Romance
Action
The Color Purple
Amadeus
Braveheart
Sense and Sensibility
Lethal Weapon
Ocean's 11
The Lion King
The Princess Diaries
Independence Day
Dumb and Dumber

Representing profiles: example

- Let us represent the users as vectors $\mathbf{p}_u$ with weights for the properties representing how much a user liked a certain property in the past
- E.g. a user who likes serious action movies might be represented with this vector

$$\mathbf{p}_1 = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

Representing profiles: example

- To express that a user dislikes a particular property we use negative weights
- A user who dislikes serious movies, i.e. a user who prefers funny (action) movies

$$\mathbf{p}_2 = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$

Representing profiles: example

- We represent items with vectors $\mathbf{q}_i$
- Weights for the properties represent the strength of funny/serious or action/romance properties in a given movie
- E.g. Braveheart is a serious action movie:

$$\mathbf{q}_1 = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Representing profiles: example

- The Color Purple is a serious romance movie

$$\mathbf{q}_2 = \begin{pmatrix} 2.5 \\ -2 \end{pmatrix}$$

Representing profiles: example

- We calculate a rating that a user gives for a certain movie by calculating $\mathbf{p}^T\mathbf{q}$
- Sum of the weight products
- The first user likes serious action movies, she rates Braveheart (a serious action) movie with 10 stars

$$\mathbf{p}_1^T\mathbf{q}_1 = \begin{pmatrix} 2 & 2 \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = 10$$

Representing profiles: example

- The second user likes funny action movies, she rates Braveheart (a serious action) movie with only 1 star

$$\mathbf{p}_2^T \mathbf{q}_1 = \begin{pmatrix} -1 & 2 \end{pmatrix} \begin{pmatrix} 3 \\ 2 \end{pmatrix} = 1$$

Representing profiles: example

- The first user likes serious action movies, she rates The Color Purple (a serious romance) movie with only 1 star

$$\mathbf{p}_1^T \mathbf{q}_2 = \begin{pmatrix} 2 & 2 \end{pmatrix} \begin{pmatrix} 2.5 \\ -2 \end{pmatrix} = 1$$

Representing users: The $\mathbf{P}$ Matrix

- We group all user vectors $\mathbf{p} \in \mathbb{R}^k$ into a matrix $\mathbf{P} \in \mathbb{R}^{n \times k}$
- n is the number of users, and k is the number of properties (factors)

$$\begin{pmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \\ \vdots \\ \mathbf{p}_n^T \end{pmatrix}$$

Representing items: The $\mathbf{Q}$ Matrix

- We group all item vectors $\mathbf{q} \in \mathbb{R}^k$ into a matrix $\mathbf{Q}^T \in \mathbb{R}^{k \times m}$
- m is the number of items, and k is the number of properties (factors)

$$\begin{pmatrix} \mathbf{q}_1 & \mathbf{q}_2 & \dots \mathbf{q}_m \end{pmatrix}$$

The product: $\mathbf{PQ}^T$

- The utility matrix $\mathbf{R}$ is given by:

$$\mathbf{R} = \mathbf{PQ}^T$$

The product: $\mathbf{PQ}^T$

- **Problem:** We do not have matrices $\mathbf{P}$ and $\mathbf{Q}$
- We do not know the weights for the factors
- **Solution:** We will learn these matrices from the data
- We will decompose $\mathbf{R}$ to obtain $\mathbf{P}$ and $\mathbf{Q}$
- Finally, we use $\mathbf{P}$ and $\mathbf{Q}$ to predict missing values in $\mathbf{R}$

Matrix Factorization

Matrix Factorization

Matrix Factorization

- We start with $\mathbf{R} \in \mathbb{R}^{n \times m}$ and want to find $\mathbf{P} \in \mathbb{R}^{n \times k}$ and $\mathbf{Q} \in \mathbb{R}^{k \times m}$ s.t. $\mathbf{P}\mathbf{Q}^T$ closely approximates $\mathbf{R}$

$$\begin{pmatrix} 5 & 2 & 4 & 4 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ 2 & & 3 & 1 & 4 \\ 2 & 5 & 4 & 3 & 5 \\ 4 & 4 & 5 & 4 & \end{pmatrix} = \begin{pmatrix} p_{11} & p_{12} \\ p_{21} & p_{22} \\ p_{31} & p_{32} \\ p_{41} & p_{42} \\ p_{51} & p_{52} \end{pmatrix} \times \begin{pmatrix} q_{11} & q_{12} & q_{13} & q_{14} & q_{15} \\ q_{21} & q_{22} & q_{23} & q_{24} & q_{25} \end{pmatrix}$$

Root-Mean-Square-Error

- We approximate $\mathbf{R} \rightarrow$ we need to measure the approximation error
- We can pick among several measures for this error
- A typical choice is the root-mean-square-error (RMSE) (or equivalently SSE as before):
  1. Sum over all non-blank entries in $\mathbf{R}$ the square of the difference between that entry and the corresponding entry in the product $\mathbf{P}\mathbf{Q}^T$
  2. Take the average of these squares by dividing by the number of terms in the sum (i.e. the number of non-blank entries in $\mathbf{R}$)
  3. Take the square root of the mean
- Minimizing the sum of the squares is equivalent to minimizing the square root of the average square, thus we can omit the last two steps

Matrix factorization

- The objective function:
- $SSE = J(\mathbf{P}, \mathbf{Q}) = \|\mathbf{P}\mathbf{Q}^T - \mathbf{R}\|_F^2 = \sum_{u,i}([\mathbf{P}\mathbf{Q}^T]_{ui} - r_{ui})^2$
- Gradient of the objective function:

$$\frac{\partial J(\mathbf{P}, \mathbf{Q})}{\partial p_{uy}} = 2 \sum_{u,i} \sum_{l} (p_{ul} q_{li} - r_{ui}) q_{iy}$$

$$\frac{\partial J(\mathbf{P}, \mathbf{Q})}{\partial q_{ix}} = 2 \sum_{u,i} \sum_{l} (p_{ul} q_{li} - r_{ui}) p_{ux}$$

Matrix factorization

- The objective function:
- $SSE = J(\mathbf{P}, \mathbf{Q}) = \|\mathbf{P}\mathbf{Q}^T - \mathbf{R}\|_F^2 = \sum_{u,i}([\mathbf{P}\mathbf{Q}^T]_{ui} - r_{ui})^2$
- Gradient of the objective function is given in the matrix form as:

$$\frac{\partial J(\mathbf{P}, \mathbf{Q})}{\partial \mathbf{P}} = 2(\mathbf{P}\mathbf{Q}^T - \mathbf{R})\mathbf{Q}$$

$$\frac{\partial J(\mathbf{P}, \mathbf{Q})}{\partial \mathbf{Q}} = 2(\mathbf{P}\mathbf{Q}^T - \mathbf{R})^T\mathbf{P}$$

Overfitting

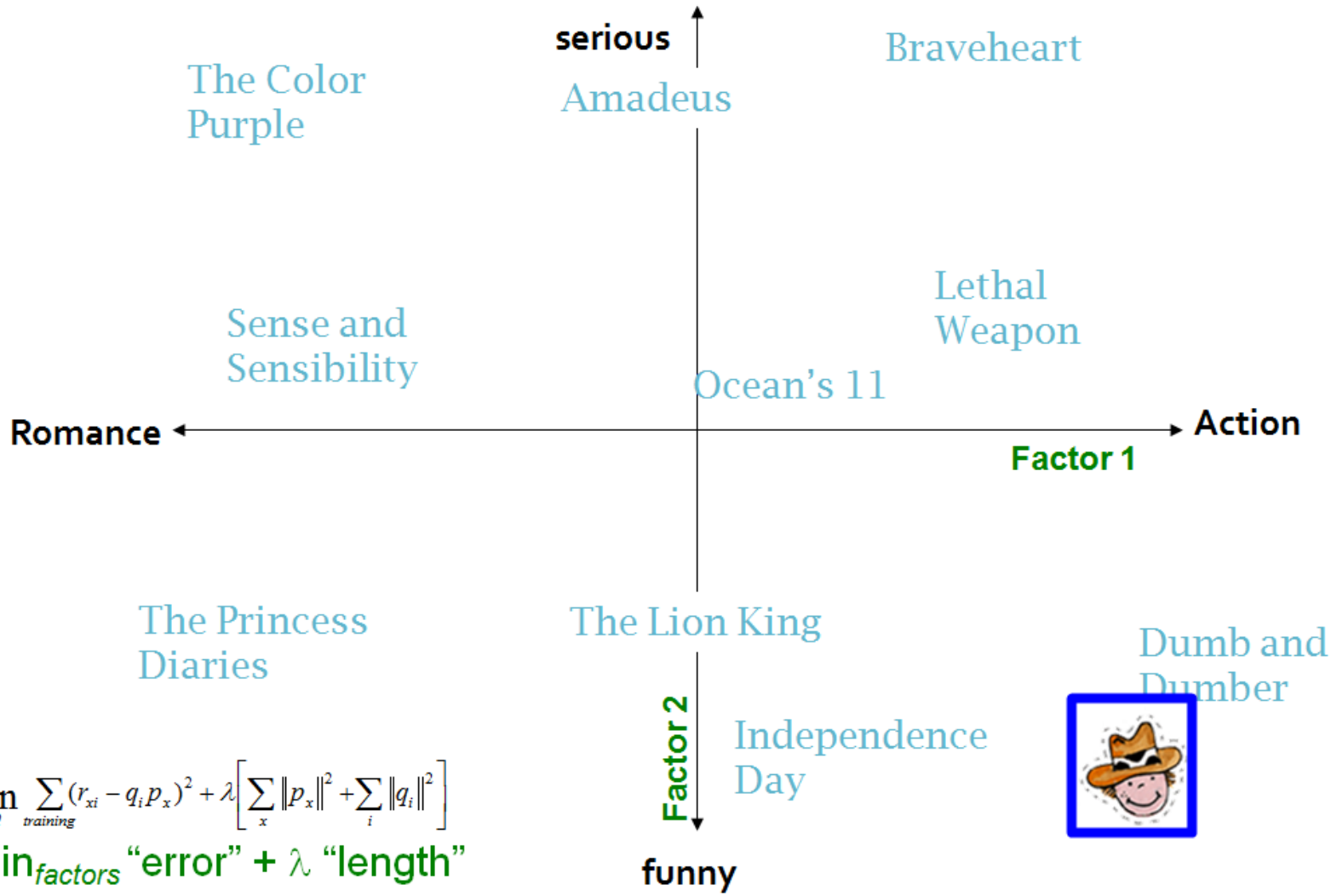
- One problem that may arise
- We arrive at a local minimum that fits very well to the given data
- But it fails to reflect the underlying process that generates the data
- In other words, the RMSE is small on the given data, but it does not do well predicting future data
- This problem is called *overfitting*

Regularization

- To solve overfitting we introduce regularization
- Allows a richer data model where there are sufficient data
- Shrinks the model aggressively where data are scarce

$$J(\mathbf{P}, \mathbf{Q}) = \sum_{u,i} ([\mathbf{P}\mathbf{Q}^T]_{ui} - r_{ui}) + \lambda_1 \sum_{u,y} p_{uy}^2 + \lambda_2 \sum_{i,x} q_{ix}^2$$

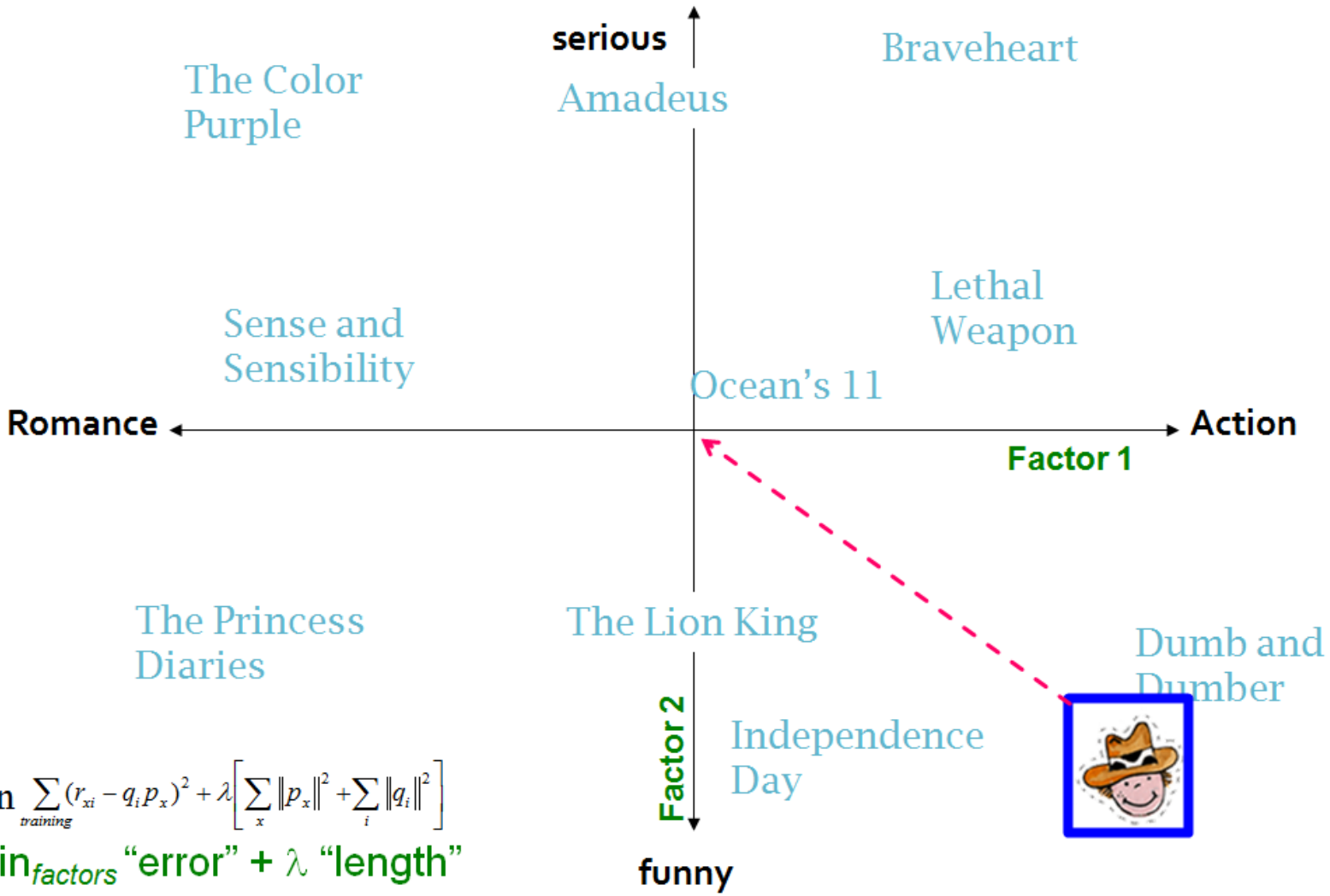
Effects of regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

$\min_{factors}$ "error" + λ "length"

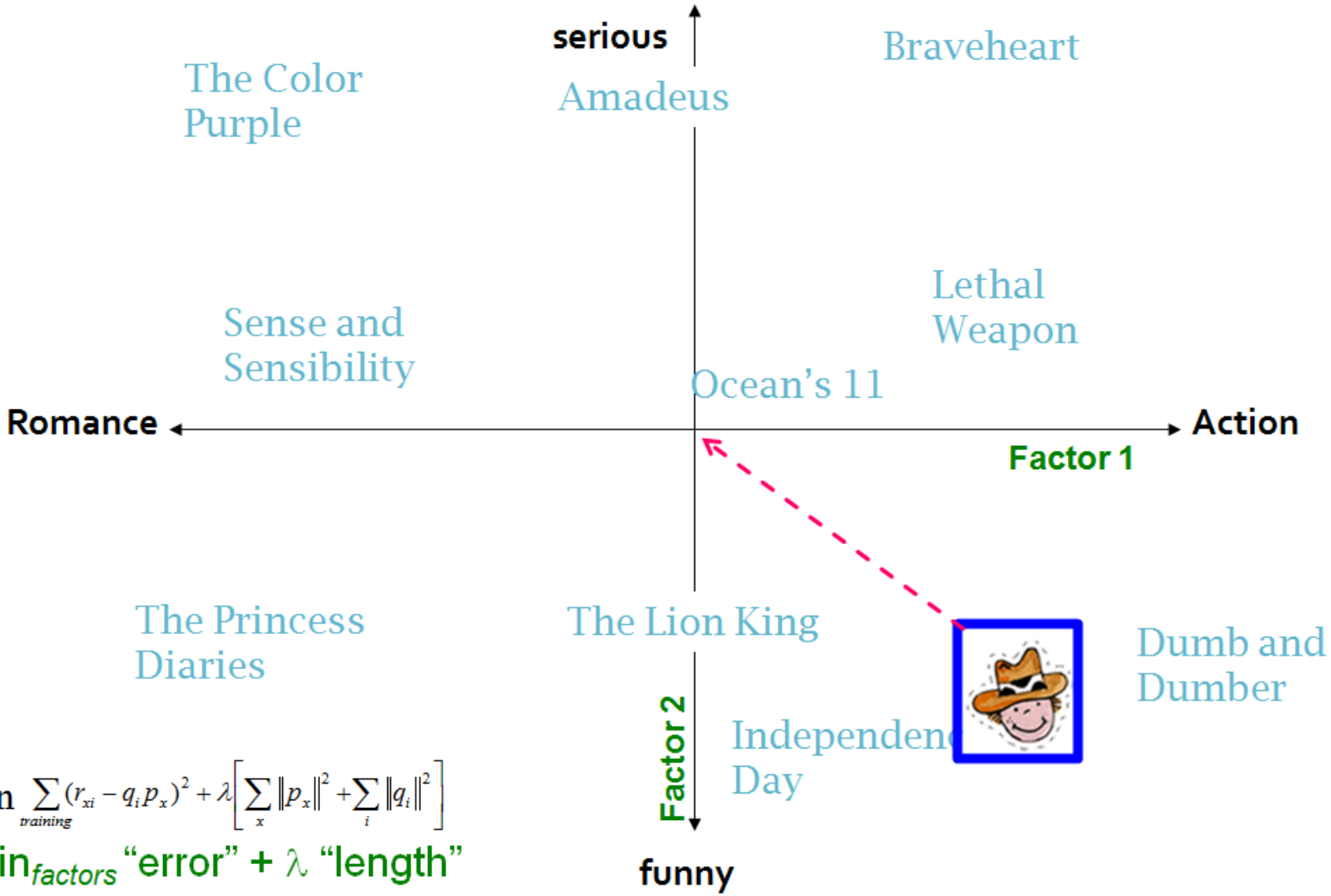
Effects of regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

$\min_{factors}$ "error" + λ "length"

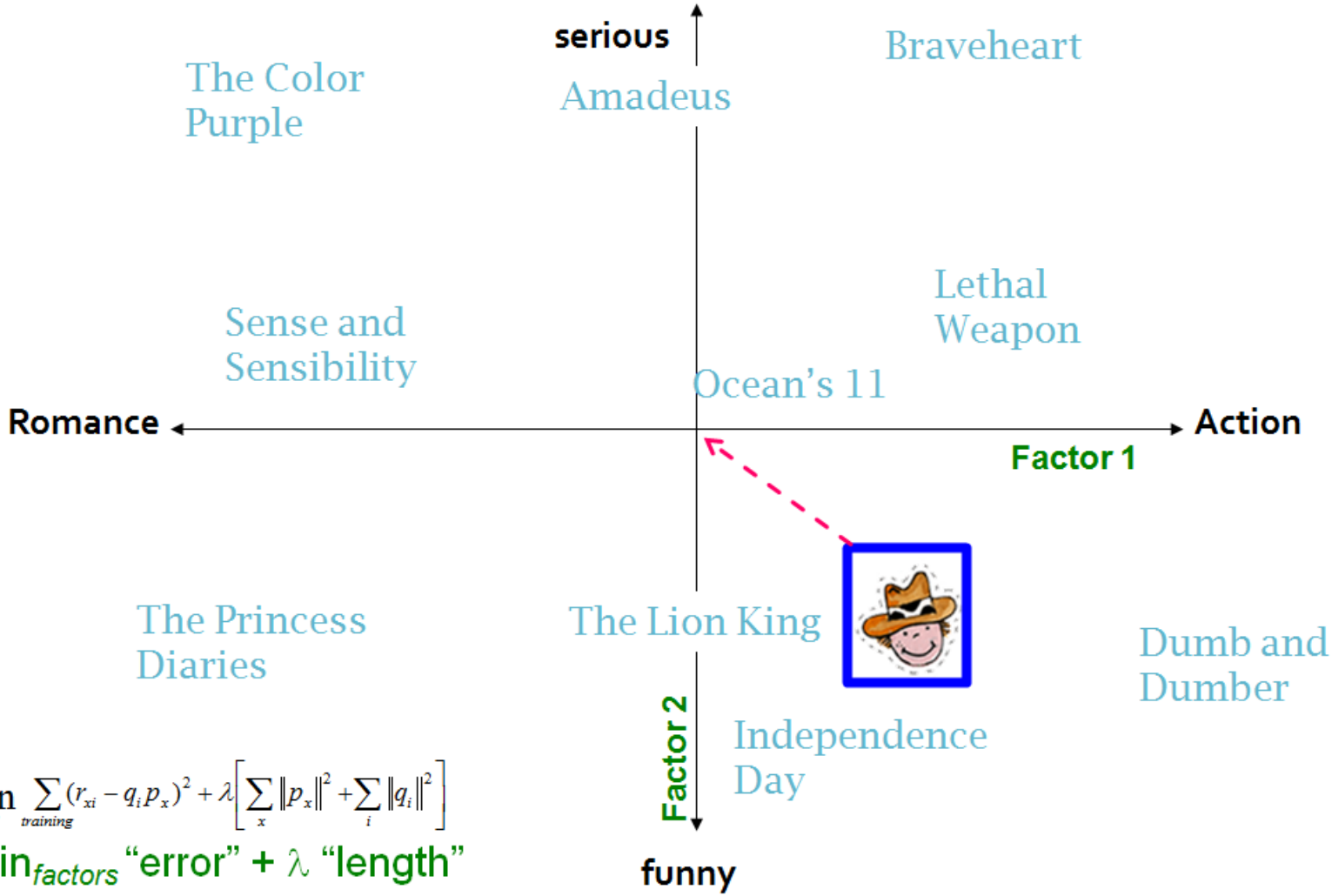
Effects of regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

$\min_{factors}$ "error" + λ "length"

Effects of regularization



$$\min_{P,Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[\sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

$\min_{factors}$ "error" + λ "length"

The Netflix Prize: Performance of various methods

- Global average: 1.1296
- User average: 1.0651
- Movie average: 1.0533
- Netflix: 0.9514
- CF: 0.94
- CF+Biases+learned weights: 0.91
- Latent factors: 0.90
- Latent factors+Biases: 0.89
- Grand Prize: 0.8563

Alternative algorithms

- Computing gradients is slow
- Some alternative approaches
- Instead of evaluating gradient over all ratings evaluate it for only ratings for a single element in a matrix
- Then iterate over the elements

Alternative algorithms

- Suppose we start with $\mathbf{P}$ and $\mathbf{Q}$ with all ones:

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix}$$

Alternative algorithms

$$\begin{pmatrix} 5 & 2 & 4 & 4 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ 2 & & 3 & 1 & 4 \\ 2 & 5 & 4 & 3 & 5 \\ 4 & 4 & 5 & 4 & \end{pmatrix} - \begin{pmatrix} 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 0 & 2 & 2 & 1 \\ 1 & -1 & 0 & 2 & -1 \\ 0 & & 1 & -1 & 2 \\ 0 & 3 & 2 & 1 & 3 \\ 2 & 2 & 3 & 2 & \end{pmatrix}$$

Alternative algorithms

- Sum of squares:
  1. Row 1: 18
  2. Row 2: 7
  3. Row 3: 6
  4. Row 4: 23
  5. Row 5: 21
- Total sum: 75

Alternative algorithms

- Suppose we start with $\mathbf{P}$ and $\mathbf{Q}$ with all ones and make a single adjustment (p_{11}):

$$\begin{pmatrix} x & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} x+1 & x+1 & x+1 & x+1 & x+1 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix}$$

Alternative algorithms

- Sum of squares:
  - Row 1: $(5-(x+1))^2+(2-(x+1))^2+(4-(x+1))^2+(4-(x+1))^2++(3-(x+1))^2$
  - This simplifies to: $(4-x)^2+(1-x)^2+(3-x)^2+(3-x)^2+(2-x)^2$
- We are looking for x that minimizes the sum:

$$\frac{ds}{dx}=0$$

Alternative algorithms

$$\frac{ds}{dx} = -2((4-x) + (1-x) + (3-x) + (3-x) + (2-x)) = 0$$

- This gives $x = 2.6$

$$\begin{pmatrix} 2.6 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 3.6 & 3.6 & 3.6 & 3.6 & 3.6 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix}$$

Alternative algorithms

- Now we would again make a single adjustment (q_{11}) and repeat the process

$$\begin{pmatrix} 2.6 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \times \begin{pmatrix} y & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 2.6y+1 & 3.6 & 3.6 & 3.6 & 3.6 \\ y+1 & 2 & 2 & 2 & 2 \\ y+1 & 2 & 2 & 2 & 2 \\ y+1 & 2 & 2 & 2 & 2 \\ y+1 & 2 & 2 & 2 & 2 \end{pmatrix}$$

Alternative algorithms

- Finding the decomposition with the minimal RMSE involves starting with some arbitrarily chosen $\mathbf{P}$ and $\mathbf{Q}$ and iteratively adapting the matrices to make the RMSE smaller
- We consider only adjustments to a single element of $\mathbf{P}$ or $\mathbf{Q}$
- In principle we could also make more complex adjustments
- In a typical example we will encounter many *local minima*
- In that case no allowable adjustments to $\mathbf{P}$ or $\mathbf{Q}$ will make the RMSE smaller

Alternative algorithms

- Only one of these will be the *global minimum*
- That is the the least possible RMSE
- To increase the chances of finding the global minimum we may start the iteration many times with different starting points
- However, there is no guarantee that we will find the global minimum

Factors: interpretation

Factors: interpretation

Factors: interpretation

- We reduced the dimension of the utility matrix to k dimensions
- These dimensions are called **factors**
- Our $\mathbf{P}$ matrix connects users with the latent factors, i.e. it is a projection of users to the latent factors space
- Our $\mathbf{Q}$ matrix connects movies with the latent factors, i.e. it is a projection of movies to the latent factors space
- These projections bring similar users/movies together

Factors: interpretation

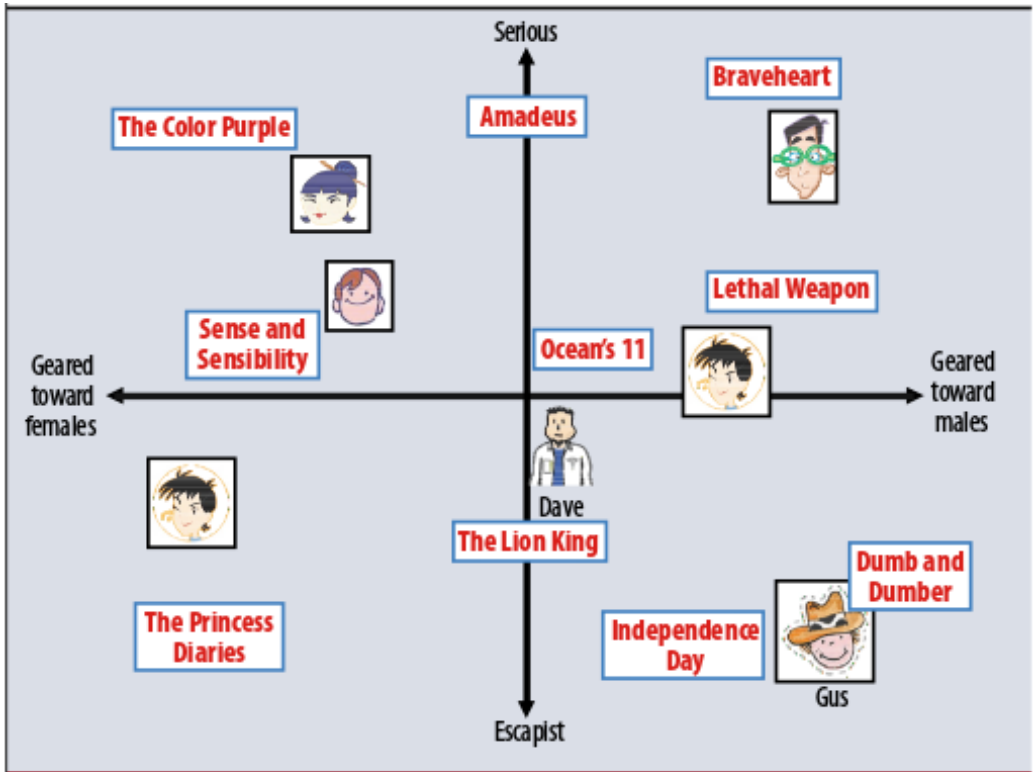


Figure 2. A simplified illustration of the latent factor approach, which characterizes both users and movies using two axes—male versus female and serious versus escapist.

- From the winners of the Netflix prize paper: http://ieeexplore.ieee.org/xpl/articleDetails.jsp?arnumber=5197422

Factors: interpretation

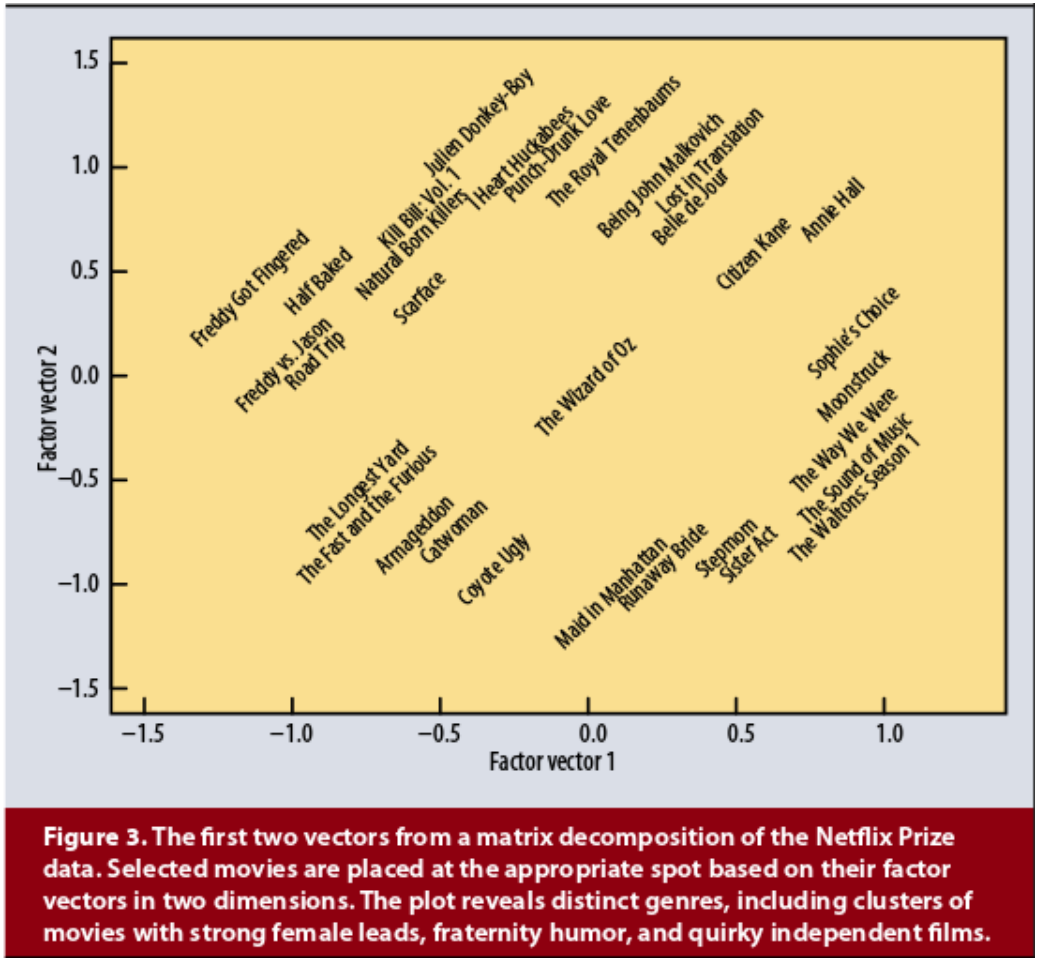
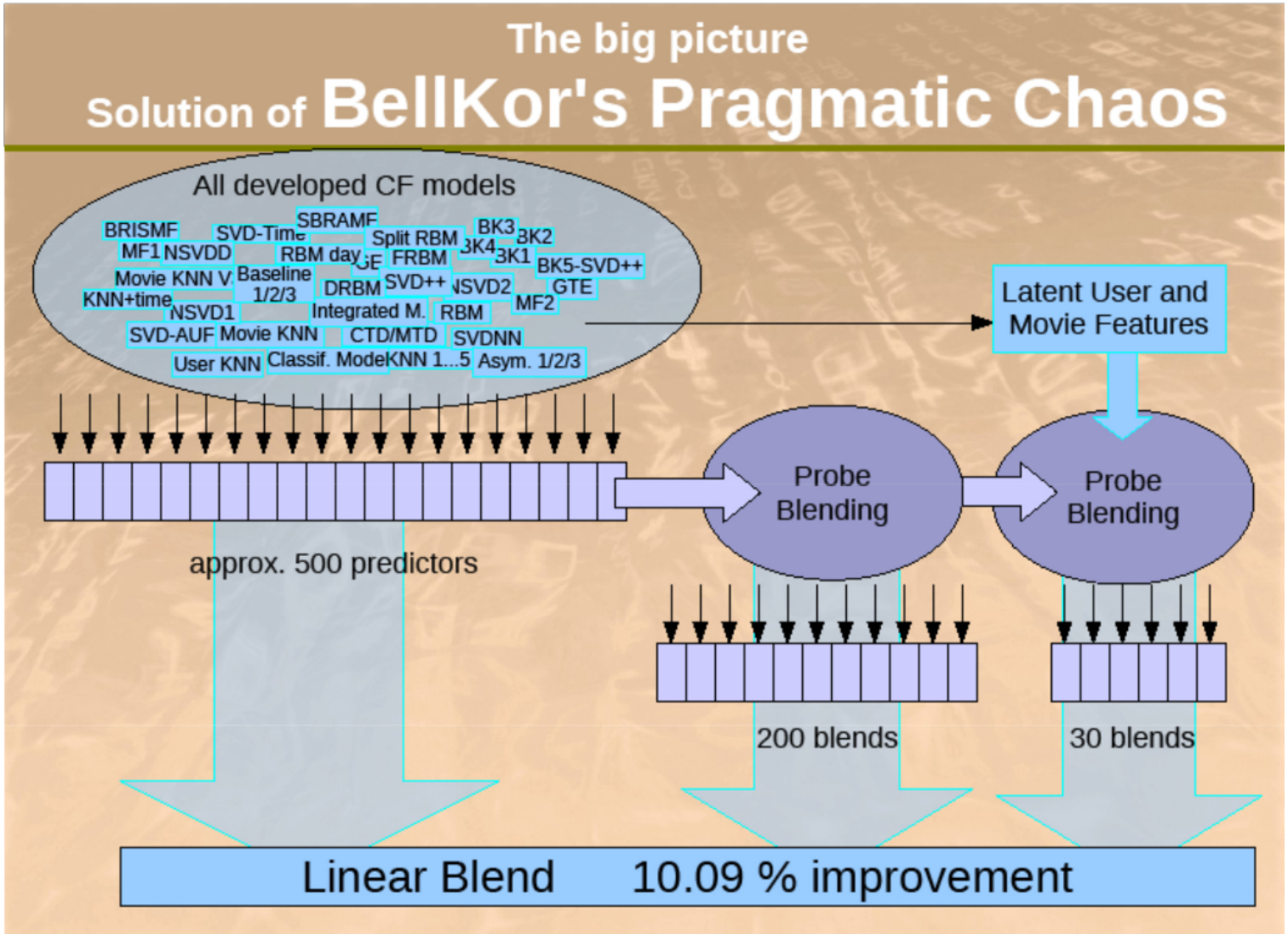


Figure 3. The first two vectors from a matrix decomposition of the Netflix Prize data. Selected movies are placed at the appropriate spot based on their factor vectors in two dimensions. The plot reveals distinct genres, including clusters of movies with strong female leads, fraternity humor, and quirky independent films.

The Netflix Prize: Performance of various methods

- Global average: 1.1296
- User average: 1.0651
- Movie average: 1.0533
- Netflix: 0.9514
- CF: 0.94
- CF+Biases+learned weights: 0.91
- Latent factors: 0.90
- Latent factors+Biases: 0.89
- Latent factors+Biases+Time: 0.876
- Grand Prize: 0.8563

BellKor solution



Theory of Matrix Factorization

Theory of Matrix Factorization

Formalizing Factorization

- We want to find $\mathbf{U} \in \mathbb{R}^{n \times k}$ and $\mathbf{V} \in \mathbb{R}^{m \times k}$
- Such that $\mathbf{U}\mathbf{V}^T$ closely approximates $\mathbf{M} \in \mathbb{R}^{n \times m}$
- If $\mathbf{M}$ is of rank k we can always decompose to obtain $\mathbf{U}$ and $\mathbf{V}$, e.g, by SVD
- On the other hand, any pair of $\mathbf{U}$ and $\mathbf{V}$ will yield a rank k matrix

Formalizing Factorization

- To account for (possible) blank elements in the matrix $\mathbf{M}$ we introduce a weighting matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$
- We work with $\mathbf{W} \odot \mathbf{M}$, where $\odot$ is element-wise (Hadamard) product of two matrices
- E.g. we set $W_{ij} = 1$ for non-blank and $W_{ij} = 0$ for blank elements in the utility matrix
- Now, we can think of the factorization problem as an **unconstrained optimization** over pairs of matrices $(\mathbf{U}, \mathbf{V})$ with the objective function:

$$J(\mathbf{U}, \mathbf{V}) = ||\mathbf{W} \odot (\mathbf{M} - \mathbf{U}\mathbf{V}^T)||_F^2$$

Non-Uniqueness

- This decomposition is not unique
- For any invertible matrix $\mathbf{R} \in \mathbb{R}^{k \times k}$ the pair $(\mathbf{UR}, \mathbf{V}(\mathbf{R}^{-1})^T)$ provides a factorization equivalent to $(\mathbf{U}, \mathbf{V})$:

$$\begin{aligned} \mathbf{UR}(\mathbf{V}((\mathbf{R}^{-1})^T)^T &= \mathbf{UR}((\mathbf{R}^{-1})^T)^T\mathbf{V}^T \\ &= \mathbf{URR}^{-1}\mathbf{V}^T \\ &= \mathbf{UV}^T \\ \implies J(\mathbf{UR}, \mathbf{V}(\mathbf{R}^{-1})^T) &= J(\mathbf{U}, \mathbf{V}) \end{aligned}$$

Ill-posed problem

- Thus, the matrix factorization problem is an example of an ill-posed problem
- Geometrically, matrix factorization assumes that data is described by a convex cone
- The problem: which cone?

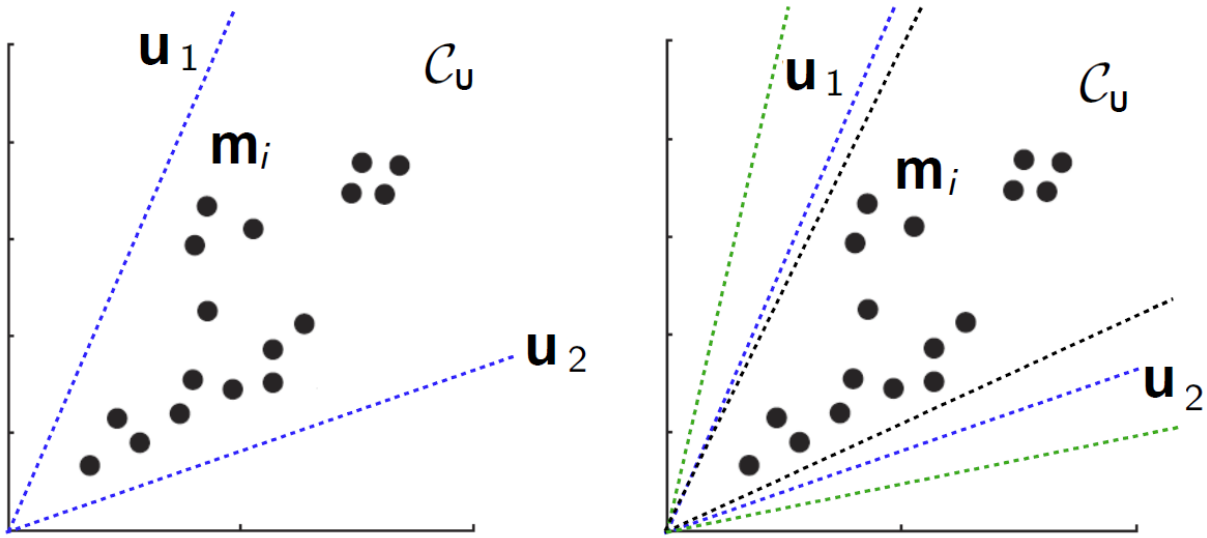


Figure: Figures adapted from "A Tutorial on Non-Negative Matrix Factorization" (Essid and Ozerov)

Ill-posed problem

- We impose constraints on the set of possible solutions to select useful solutions
- Useful depends on the application domain and our overall goal
- For example, orthogonality constraints, i.e. $\mathbf{V}^T\mathbf{V} = \mathbf{I}$ and $\mathbf{U}$ diagonal
- Non-negativity constraints, i.e., all elements in $\mathbf{V}$ and $\mathbf{U}$ greater or equal to zero
- Sparsity constraint, i.e., only few elements in $\mathbf{V}$ and $\mathbf{U}$ are not equal to zero

Weighted case

- W_{ij} arbitrary, e.g. $W_{ij} = 1$ for non-blank and $W_{ij} = 0$ for missing elements
- The objective function:

$$J(\mathbf{U}, \mathbf{V}) = ||\mathbf{W} \odot (\mathbf{M} - \mathbf{U}\mathbf{V}^T)||_F^2$$

- Partial derivatives:

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{U}} &= 2(\mathbf{W} \odot (\mathbf{U}\mathbf{V}^T - \mathbf{M}))\mathbf{V} \\ \frac{\partial J}{\partial \mathbf{V}} &= 2(\mathbf{W} \odot (\mathbf{U}\mathbf{V}^T - \mathbf{M})^T)\mathbf{U} \\ &= 2(\mathbf{W} \odot (\mathbf{V}\mathbf{U}^T - \mathbf{M}^T))\mathbf{U} \end{aligned}$$

Weighted case

- Due to the non uniform weights we can not obtain a closed form solution and need to solve the equations numerically
- The equation $\frac{\partial J}{\partial \mathbf{U}} = 0$ is still a linear system in $\mathbf{U}$ for a fixed $\mathbf{V}$
- Similarly, the equation $\frac{\partial J}{\partial \mathbf{V}} = 0$ is a linear system in $\mathbf{V}$ for a fixed $\mathbf{U}$
- However, jointly $J(\mathbf{U}, \mathbf{V})$ is non-convex function and for non uniform weights local minima emerge

Weighted case

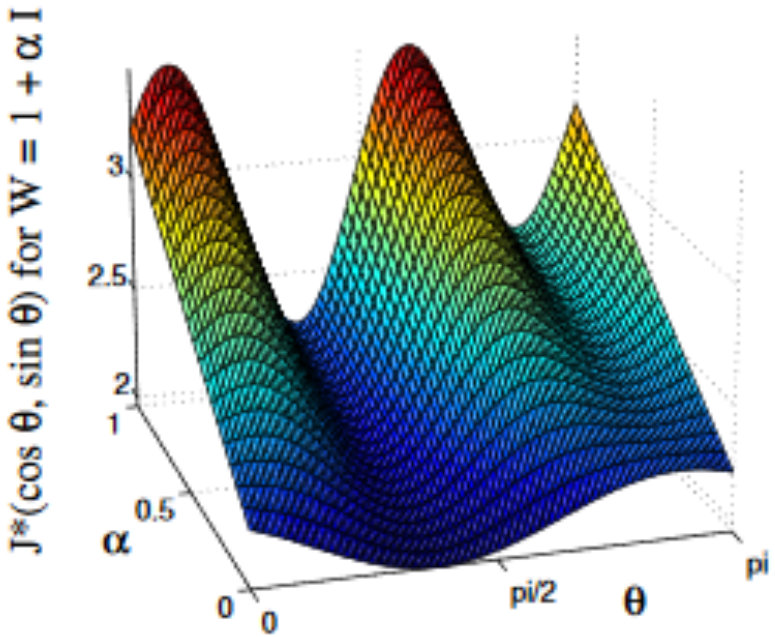


Figure: Emergence of a local minima with non-uniform weights from "Weighted Low-Rank Approximations" (Srebro and Jaakkola)

Biconvex problem

- Matrix factorization is an instance of biconvex optimization problems
- Biconvex: we fix a subset of parameters A and the objective function is convex for the remaining subset of parameters B
- And vice versa
- Thus, we fix A and optimize B and then alternate between A and B
- In matrix factorization we fix $\mathbf{V}$ optimize for $\mathbf{U}$ and then alternate

Alternating least square (ALS) algorithm

- Subproblem for $\mathbf{U}$ (for a given $\mathbf{V}$):

$$J_{\mathbf{U}}(\mathbf{U}) = ||\mathbf{M} - \mathbf{U}\mathbf{V}^T||_F^2$$

- Subproblem for $\mathbf{V}$ (for a given $\mathbf{U}$):

$$J_{\mathbf{V}}(\mathbf{V}) = ||\mathbf{M} - \mathbf{U}\mathbf{V}^T||_F^2$$

- We omit weights for simplicity

Alternating least square (ALS) algorithm

- Solution for $\mathbf{U}$:

$$\begin{aligned} \frac{dJ_{\mathbf{U}}}{d\mathbf{U}} &= 2(\mathbf{U}\mathbf{V}^T - \mathbf{M})\mathbf{V} \\ 2(\mathbf{U}\mathbf{V}^T - \mathbf{M})\mathbf{V} &= 0 \\ \mathbf{U}\mathbf{V}^T\mathbf{V} &= \mathbf{M}\mathbf{V} \\ \mathbf{U} &= \mathbf{M}\mathbf{V}(\mathbf{V}^T\mathbf{V})^{-1} \end{aligned}$$

Alternating least square (ALS) algorithm

- Solution for $\mathbf{V}$:

$$\begin{aligned} \frac{dJ_{\mathbf{V}}}{V} &= 2(\mathbf{U}\mathbf{V}^T - \mathbf{M})^T\mathbf{U} \\ 2(\mathbf{V}\mathbf{U}^T - \mathbf{M}^T)\mathbf{U} &= 0 \\ \mathbf{V}\mathbf{U}^T\mathbf{U} &= \mathbf{M}^T\mathbf{U} \\ \mathbf{V} &= \mathbf{M}^T\mathbf{U}(\mathbf{U}^T\mathbf{U})^{-1} \end{aligned}$$

ALS: complete algorithm

- Initialize $\mathbf{V}$ e.g. randomly
- Iterate until convergence or a given threshold is reached:
  1. Compute $\mathbf{U} = \mathbf{M}\mathbf{V}(\mathbf{V}^T\mathbf{V})^{-1}$
  2. With newly computed $\mathbf{U}$ compute $\mathbf{V} = \mathbf{M}^T\mathbf{U}(\mathbf{U}^T\mathbf{U})^{-1}$